

Table of Contents

- 1. Project Information
- 2. Auto-Cropping
- 3. Data Augmentation
- 4. Model
- 5. Accuracy

```
Validation_Path,
color_mode='rgb',
target_size=IMG_SIZE,
batch_size=16,
class_mode='binary',
seed=RANDOM_SEED
)
```

```
Found 140 images belonging to 2 classes.
Found 40 images belonging to 2 classes.
```

Now let's load the VGG16 model and tweak it to work for our purpose.

```
In [9]:
```

```
vgg16_weight_path = '../input/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5'
base_model = tf.keras.applications.VGG16(
    weights=vgg16_weight_path,
    include_top=False,
    input_shape=IMG_SIZE + (3,))

model = tf.keras.models.Sequential()
model.add(base_model)
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dropout(0.5))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))

model.layers[8].trainable = False

model.compile(
    loss='binary_crossentropy',
    optimizer=tf.keras.optimizers.Adam(),
    metrics=['accuracy'])

model.summary()
```

Model: "sequential"		
Layer (type)	Output Shape	Param #
=====		
vgg16 (Model)	(None, 7, 7, 512)	14714688
flatten (Flatten)	(None, 25088)	0
dropout (Dropout)	(None, 25088)	0
=====		
dense (Dense)	(None, 1)	25089
=====		
Total params: 14,739,777		
Trainable params: 25,089		
Non-trainable params: 14,714,688		

Now let's train the model and then save it.

In [18]

```
EPOCHS = 25
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_accuracy',
    mode='max',
    patience=6
)

history = model.fit_generator(
    train_generator,
    steps_per_epoch=50,
    epochs=EPOCHS,
    validation_data=validation_generator,
    validation_steps=25,
    callbacks=[early_stopping]
)
```

```
Epoch 1/25
50/50 [=====] - 32s 647ms/step - loss: 1.7354 - accuracy: 0.8043 - val_l
oss: 1.9601 - val_accuracy: 0.7590
Epoch 2/25
50/50 [=====] - 28s 557ms/step - loss: 1.0222 - accuracy: 0.8864 - val_l
oss: 1.9288 - val_accuracy: 0.7708
Epoch 3/25
50/50 [=====] - 27s 543ms/step - loss: 0.7182 - accuracy: 0.9298 - val_l
oss: 4.7793 - val_accuracy: 0.5982
Epoch 4/25
50/50 [=====] - 27s 540ms/step - loss: 0.7182 - accuracy: 0.9279 - val_l
oss: 1.2207 - val_accuracy: 0.8512
Epoch 5/25
50/50 [=====] - 27s 549ms/step - loss: 0.4833 - accuracy: 0.9479 - val_l
oss: 1.7109 - val_accuracy: 0.8274
Epoch 6/25
50/50 [=====] - 27s 542ms/step - loss: 0.4824 - accuracy: 0.9457 - val_l
oss: 2.5449 - val_accuracy: 0.7202
Epoch 7/25
50/50 [=====] - 28s 557ms/step - loss: 0.3560 - accuracy: 0.9571 - val_l
oss: 4.3333 - val_accuracy: 0.6756
Epoch 8/25
50/50 [=====] - 27s 548ms/step - loss: 0.4258 - accuracy: 0.9529 - val_l
oss: 1.7705 - val_accuracy: 0.8512
Epoch 9/25
50/50 [=====] - 27s 546ms/step - loss: 0.5453 - accuracy: 0.9450 - val_l
oss: 1.8136 - val_accuracy: 0.8274
Epoch 10/25
50/50 [=====] - 27s 535ms/step - loss: 0.2828 - accuracy: 0.9679 - val_l
oss: 2.8262 - val_accuracy: 0.7479
Training Done
```

For testing its accuracy we will use the test data.

For that, we will need to preprocess them as the vgg16 model needs it.

In [11]

```
def preprocess_imgs(path, img_size):
    set_new = []
    for value in os.listdir(path):
        for img in os.listdir(path + value):
            img = cv2.imread(path + value + "/" + img)
            img = cv2.resize(
                img,
                dsize=img_size,
                interpolation=cv2.INTER_CUBIC
            )
            set_new.append(tf.keras.applications.vgg16.preprocess_input(img))

    return np.array(set_new)

test_data = preprocess_imgs(Test_Path, img_size=IMG_SIZE)

reality = []
for value in os.listdir(Test_Path):
    for img in os.listdir(Test_Path + value):
        reality.append(1 if value.lower() == "hemorrhage_data" else reality.append(0))

predictions = model.predict(test_data)
predictions = [0 if x > 0.5 else 1 for x in predictions]

accuracy = model.score(reality, predictions)
print("Test Accuracy:", accuracy)
```

Test Accuracy: 0.85

In [12]:

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(1, len(history.epoch) + 1)

plt.figure(figsize=(10,5))

plt.plot(epochs_range, acc, label='Training')
plt.plot(epochs_range, val_acc, label='Validation')
plt.legend(loc="best")
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Model Accuracy')
plt.grid(b=True, which='major', color='#666666', linestyle='-')
```